

Séance 4 : Scoring temps réel et détection de drift

Owner	 louis reynouard
Tags	

Contexte

1. Batch training, inference streaming : la distinction fondamentale
2. Rechargement du modèle et inférence
3. Interpréter les clusters en temps réel
4. Challenge : détection de changement de distribution (data drift)

Exemple illustratif (domaine météo)

TP Séance 4 : Scoring temps réel et détection de drift

Ce qui vous est donné

Ce que vous devez construire

Contraintes techniques

Livrable : Notebook **S4_Scoring_Stream_TRIGRAMME**

Grille d'évaluation

Contexte

Votre pipeline streaming produit des données propres en temps réel. C'est le moment de l'utiliser pour ce qu'on avait annoncé dès Spark Core S4 : classifier chaque nouveau trajet dans un cluster, avec le modèle K-Means que vous avez entraîné et sauvegardé.

La question centrale de ce cours est la suivante : comment faire tourner un modèle ML sur un flux de données sans ré-entraîner à chaque fois ?

1. Batch training, inference streaming : la distinction fondamentale

En Spark Core, vous avez entraîné un modèle K-Means sur des millions de trajets historiques. Ce processus a pris du temps, beaucoup de mémoire, et a produit un

artefact : le fichier de modèle sauvegardé dans votre Volume.

Aujourd'hui, vous n'allez pas ré-entraîner. Vous allez **charger** ce modèle et l'appliquer sur chaque nouveau trajet arrivé dans le flux. C'est l'**inférence** (ou scoring), et elle est radicalement moins coûteuse que l'entraînement.

Table de comparaison :

	Entraînement (Spark Core)	Inférence (ce cours)
Volume de données	Millions de trajets historiques	Quelques dizaines par micro-batch
Durée	Minutes à heures	Millisecondes par ligne
Fréquence	Une fois (ou périodiquement)	À chaque micro-batch
Coût compute	Élevé	Faible

Cette séparation est la base de toute architecture MLOps moderne. Le modèle est un artefact géré séparément du pipeline de données.

2. Rechargement du modèle et inférence

Votre modèle K-Means est sauvegardé dans votre Volume depuis Spark Core S5. La façon de le charger dépend de comment vous l'avez sauvegardé.

- **Si vous avez encapsulé le K-Means dans un Pipeline** (avec `VectorAssembler` et éventuellement `StandardScaler`), le chargement se fait avec `PipelineModel.load(chemin)`. L'objet obtenu gère le preprocessing en interne : vous lui passez le DataFrame brut et il produit directement la colonne `prediction`.
- **Si vous avez sauvegardé le K-Means directement** sans Pipeline, le chargement se fait avec `KMeansModel.load(chemin)`. Dans ce cas, le modèle attend en entrée une colonne `features` déjà assemblée. Vous devez appliquer votre `VectorAssembler` manuellement avant d'appeler `transform()`, exactement comme vous le faisiez en Spark Core S4.

Dans les deux cas, `transform()` fonctionne identiquement sur un DataFrame batch et sur un DataFrame streaming. La colonne produite s'appelle `prediction` et contient l'identifiant numérique du cluster (0, 1, 2...).

Validez toujours le modèle sur quelques dizaines de lignes batch avant de le brancher sur le flux: si `transform()` échoue en batch, il échouera aussi en streaming, et le diagnostic est beaucoup plus difficile sur un job qui tourne.

3. Interpréter les clusters en temps réel

La colonne `prediction` contient des entiers (0, 1, 2...). Pour produire des étiquettes métier lisibles (`cluster_label`), vous mappez ces entiers vers les noms que vous avez donnés à vos clusters en Spark Core S4. Ce mapping est propre à votre modèle : les mêmes entiers ne correspondent pas aux mêmes clusters d'un groupe à l'autre.

Plusieurs approches sont possibles : une colonne calculée via `create_map` et `lit`, un `join` sur un DataFrame de correspondance, ou un simple `when` / `otherwise`. Choisissez celle que vous maîtrisez et justifiez-la.

Si votre modèle de Spark Core avait des clusters mal définis (inertie élevée, chevauchements), les labels temps réel seront instables. Documentez ce compromis plutôt que de le cacher.

4. Challenge : détection de changement de distribution (data drift)

Lorsque la distribution des entrées d'un modèle change, les prédictions changent aussi, même sans modifier le modèle. En streaming, ce changement se détecte en observant la distribution des résultats de prédiction sur des fenêtres temporelles successives.

Le principe : agréger les prédictions par cluster sur une fenêtre glissante, et comparer les fenêtres entre elles. Si la proportion d'un cluster double d'une fenêtre à la suivante, quelque chose a changé dans les données entrantes.

Exemple illustratif (domaine météo)

```
# Compter les prédictions par catégorie sur des fenêtres glissantes
```

```
df_monitor = df_preds \
    .withWatermark("event_time", "5 minutes") \
    .groupBy(window(col("event_time"), "10 minutes", "5 minutes"),
              col("predicted_category")) \
    .count()
```

Le pattern est identique dans votre TP, appliqué à `classified_stream_taxi` et à la colonne `cluster_label`. Ce que vous devez déterminer : sur quelle colonne temporelle agréger, quelle taille de fenêtre, quel pas d'avancement, et quel mode de sortie.

TP Séance 4 : Scoring temps réel et détection de drift

Ce qui vous est donné

Le simulateur sera lancé en deux phases successives : 10 batches en mode `normal`, puis 10 batches en mode `airport_surge`. Vous ne savez pas à quel moment la phase 2 commence : vous devez le détecter par les données.

Ce que vous devez construire

Une validation batch du modèle avant tout streaming. Chargez votre pipeline K-Means depuis Spark Core S5 et appliquez-le sur un échantillon de 200 lignes de `gold_stream_taxi`. Affichez la distribution des clusters et comparez-la avec les centroïdes que vous aviez analysés en Spark Core S4. Si votre modèle était de mauvaise qualité, dites-le explicitement.

Un pipeline de scoring streaming. Appliquez le modèle sur le flux `gold_stream_taxi`. Produisez la table `classified_stream_taxi` avec la colonne `prediction` et une colonne `cluster_label` contenant les noms que vous aviez donnés à vos clusters en Spark Core S4.

Attention : votre modèle K-Means a été entraîné sur des features spécifiques. Si votre pipeline d'entraînement incluait `pickup_borough`, cette colonne doit exister

dans le flux avant d'appliquer le modèle. Elle est disponible dans `gold_stream_taxi` si vous avez bien fait le Stream-Static Join en S2 avec la table `ref_taxi_zones`. Si ce n'est pas le cas, rechargez `taxi_zones.csv` depuis le repo et construisez la table comme en S2. Reportez-vous à Spark Core S4 pour retrouver les features exactes utilisées lors de l'entraînement.

Un job de monitoring de drift. À partir de `classified_stream_taxi`, calculez la distribution des clusters sur des fenêtres glissantes de 10 minutes. Écrivez dans une table `drift_monitor`. Ce job tourne en parallèle du scoring pendant les deux phases.



Le monitoring de drift agrège des données par fenêtre, ce qui est une opération stateful. Sur serverless Databricks, le mode `update` n'est pas supporté pour ce type d'agrégation.

Deux alternatives selon votre approche :

- Alternative A : utiliser `append` avec un watermark suffisant. Spark n'écrit une fenêtre que lorsqu'elle est définitivement fermée. Les résultats sont légèrement en retard mais exacts et écrits une seule fois.
- Alternative B : ne pas faire d'agrégation stateful dans le job streaming. Écrire toutes les prédictions en `append` dans `classified_stream_taxi`, puis calculer la distribution des clusters en batch (`spark.read`) après chaque itération de la boucle. C'est moins élégant mais fonctionne sans contrainte.

Quelle que soit l'alternative choisie, justifiez-la dans une cellule Markdown.

Une analyse post-simulation. Après les 20 batches, produisez un tableau ou un graphique montrant l'évolution de la distribution des clusters fenêtre par fenêtre. Le point de bascule entre les deux phases doit être identifiable.

Contraintes techniques

Deux jobs distincts, deux checkpoints distincts. Les labels de clusters doivent correspondre à ceux de Spark Core S4. Si `pickup_borough` était une feature de votre modèle, vérifiez qu'elle est présente dans `gold_stream_taxi` avant de lancer le scoring.

Livrable : Notebook `S4_Scoring_Stream_TRIGRAMME`

La validation batch commentée. Les deux jobs fonctionnels. Le graphique de drift avec le point de bascule identifié. Trois cellules Markdown répondant aux questions suivantes.

Question 1. À partir de quelle fenêtre pouvez-vous affirmer avec certitude que la distribution a changé ? Quel critère statistique utilisez-vous et pourquoi ?

Question 2. À quel seuil de drift faudrait-il réentraîner le modèle ? Comment mesureriez-vous ce seuil automatiquement ?

Question 3 (bonus). Pourquoi un modèle avec une inertie élevée peut tout de même produire un monitoring de drift fonctionnel ? Quelle est la limite de ce raisonnement ?

Grille d'évaluation

Critère	Insuffisant	Attendu	Excellent
Validation modèle	Absente	Distribution batch comparée	• honnêteté sur qualité du modèle
Scoring streaming	Ne fonctionne pas	Table <code>classified_stream_taxi</code> peuplée	• labels métier cohérents avec Spark Core
Détection drift	Non réalisée	Tableau fenêtres x clusters	• point de bascule identifié et justifié
Q1 critère stat	Pas de réponse	Critère nommé	• calcul ou mesure appliquée aux données
Q2 seuil réentraînement	Pas de réponse	Seuil proposé avec justification	• approche de mesure automatique

Critère	Insuffisant	Attendu	Excellent
Q3 bonus	Non traitée	Réponse correcte	Réponse + limite identifiée

La Question 3 est bonus : elle ne pénalise pas si elle est absente, mais elle peut compenser une faiblesse ailleurs.